Variadic and Deferred

1. Variadic Functions (Variable Number of Arguments)

Sometimes you need a function to accept an unknown number of inputs (like a function that calculates the sum or maximum of any given list of numbers). Instead of forcing the user to pass a pre-built array or slice, Go allows you to create a **Variadic Function**.

- **The Syntax (...):** You place an **ellipsis ...** directly before the parameter's type.

```
func getMax(vals ...int) int { }
```

- **How it works internally:** Inside the function, Go automatically takes all the loose comma-separated arguments you passed in and packs them neatly into a **standard Slice**. You can then iterate over vals using a normal for...range loop!

The Slice Unpacking Trick: What if you already have a slice of data (e.g., mySlice := []int{1, 3, 6, 4}) and you want to pass it into a variadic function? You can't just drop the slice in, because the function is expecting loose integers.

- The Fix: You use the ellipsis again when calling the function to "unpack" the slice!
- Example: getMax(mySlice...)

2. The defer Keyword (Clean-Up Magic)

The **defer** keyword is one of Go's most beloved features. It allows you to schedule a function call to run at the very end of the surrounding function, just before it returns.

- **How it works: defer fmt.Println("Bye")**
- **Why use it?** In backend programming, if you open a database connection or a file, you must remember to close it so you don't leak memory. With **defer**, you can place the **"close"** command immediately right under the **"open"** command. Go will wait and execute that **"close"** command for you right as the function finishes, ensuring you never forget!

3. The "Gotcha": Deferred Argument Evaluation

This is a classic "Gotcha" that catches many developers (and is a very common Go interview question). When you use defer, the execution of the function is delayed, **but the arguments are evaluated immediately.**

```
i := 1
defer fmt.Println(i + 1) // Go looks at 'i' right NOW. 1 + 1 = 2. It locks in the '2'.

i++ // 'i' is now 2
fmt.Println("Hello")

// Function ends. The deferred print '2', NOT '3'!
```

Because Go evaluated **i + 1** at the exact moment it read the defer line, it saved the result (2) to use later. It completely ignores the fact that "**i"** was incremented later in the code.

Expert Takeaway

Using defer is a standard best practice for resource management in Go. If you are writing a script that fetches data from an API, your code will look like this: resp, err := http.Get("url") defer resp.Body.Close() By putting the cleanup code on the very next line, your code remains clean, readable, and perfectly safe from memory leaks, no matter how many if statements or return paths happen later in the function.